

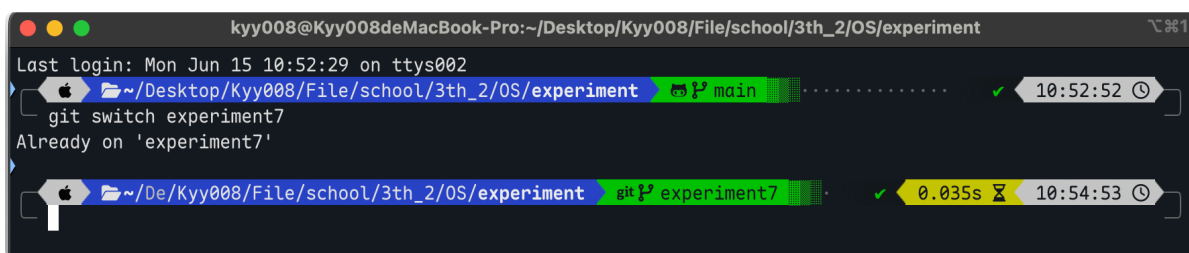
2026 春《操作系统》课程实验报告

实验 7

23301037 客昱阳

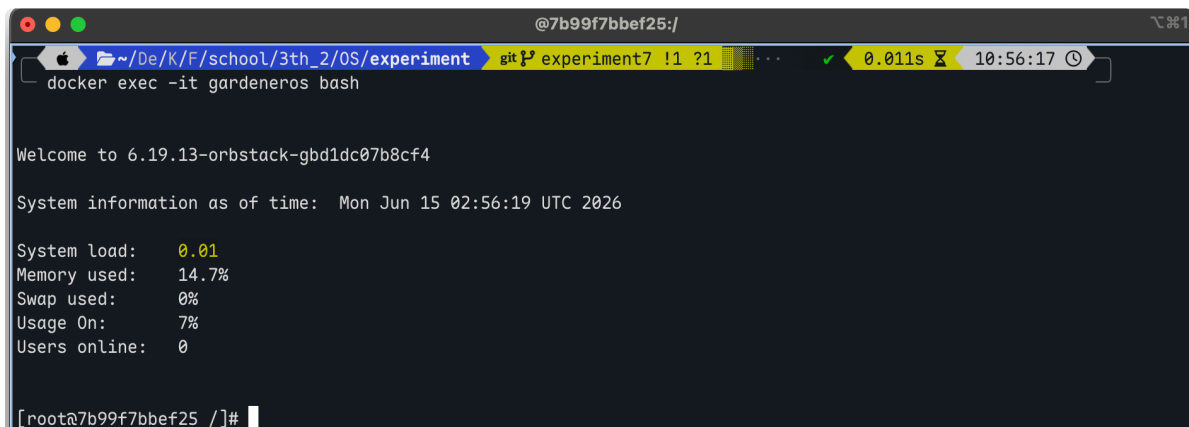
一.准备工作

- 将分支切换到实验四所需的 `experiment7` 分支



```
kyy008@Kyy008deMacBook-Pro:~/Desktop/Kyy008/File/school/3th_2/OS/experiment
Last login: Mon Jun 15 10:52:29 on ttys002
~/Desktop/Kyy008/File/school/3th_2/OS/experiment main ..... 10:52:52
git switch experiment7
Already on 'experiment7'
~/De/Kyy008/File/school/3th_2/OS/experiment git experiment7 0.035s 10:54:53
```

- 进入容器



```
@7b99f7bbef25:/
~/De/K/F/school/3th_2/OS/experiment git experiment7 !1 ?1 ... 0.011s 10:56:17
docker exec -it gardeneros bash

Welcome to 6.19.13-orbstack-gbd1dc07b8cf4

System information as of time: Mon Jun 15 02:56:19 UTC 2026

System load: 0.01
Memory used: 14.7%
Swap used: 0%
Usage On: 7%
Users online: 0

[root@7b99f7bbef25 /]#
```

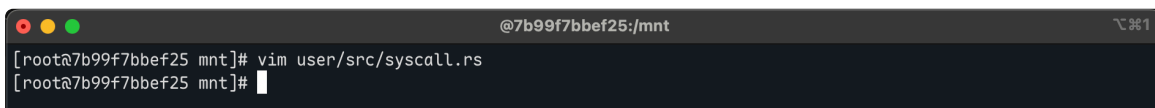
二.实验步骤

由于实验较长，步骤不给出具体代码，详情见 git 仓库。

1.修改应用程序

(1)增加重要的系统调用

- 修改 `user/src/syscall.rs`，增加进程管理相关系统调用编号和封装函数。



```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/syscall.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `user/src/lib.rs`，把新增系统调用封装成应用程序可以直接调用的函数。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/lib.rs
[root@7b99f7bbef25 mnt]#
```

(2)实现用户初始程序 initproc

- 创建用户初始程序 `initproc.rs`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/bin/initproc.rs
[root@7b99f7bbef25 mnt]#
```

(3)实现 shell 程序

- 修改 `user/src/console.rs`，封装从标准输入读取一个字符的函数 `getchar()`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/console.rs
[root@7b99f7bbef25 mnt]# vim user/src/console.rs
[root@7b99f7bbef25 mnt]#
```

- 创建用户 shell 程序 `user_shell.rs`。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/bin/user_shell.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `user/src/lib.rs`，为用户库增加动态内存分配支持

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/lib.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `user/Cargo.toml`，增加用户库堆分配器依赖

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/Cargo.toml
[root@7b99f7bbef25 mnt]#
```

2.在内核中增加系统调用

- 修改 `os/src/syscall/mod.rs`，增加 `read`、`getpid`、`fork`、`exec`、`waitpid` 系统调用编号和分发

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/syscall/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/syscall/fs.rs`，实现 `sys_read` 系统调用

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/syscall/fs.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/syscall/process.rs`，引入进程管理系统调用需要的模块

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/syscall/process.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/syscall/process.rs`，实现 `getpid`、`fork`、`exec`、`waitpid` 系统调用

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/syscall/process.rs
[root@7b99f7bbef25 mnt]#
```

3.应用的链接与加载

(1)基于名字的应用链接

- 修改 `os/build.rs`，在生成的 `link_app.S` 中加入应用程序名称表 `_app_names`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/build.rs
[root@7b99f7bbef25 mnt]#
```

(2)基于名字的应用加载

- 修改 `os/src/loader.rs`，增加应用程序名称表 `APP_NAMES`，并支持通过应用名获取 ELF 数据。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/loader.rs
[root@7b99f7bbef25 mnt]#
```

4.进程标识符与内核栈

(1)实现进程标识符

- 创建 `os/src/task/pid.rs`，实现进程标识符 `PidHandle` 和 `PID` 分配器。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/pid.rs
[root@7b99f7bbef25 mnt]#
```

(2)在内核栈中保存进程标识符

- 继续修改 `os/src/task/pid.rs`，实现 `KernelStack`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/pid.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/mm/memory_set.rs`，为 `MemorySet` 增加根据起始虚拟页号移除逻辑段的方法。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

5.修改实现进程控制块

- 修改 `os/src/task/task.rs`，引入进程控制块需要的模块

```
[root@7b99f7bbef25 mnt]# vim os/src/task/task.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/task/task.rs`，重新定义 `TaskStatus`、`TaskControlBlock` 和 `TaskControlBlockInner`

```
[root@7b99f7bbef25 mnt]# vim os/src/task/task.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/task/task.rs`，为 `TaskControlBlockInner` 实现基本方法

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/task.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/task/task.rs`，重新实现 `TaskControlBlock` 的基本方法和 `new()`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/task.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/task/task.rs`，实现 `exec()`、`fork()` 和 `getpid()`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/task.rs
[root@7b99f7bbef25 mnt]#
```

6.实现任务管理器

- 创建 `os/src/task/manager.rs`，实现就绪队列形式的任务管理器。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/manager.rs
[root@7b99f7bbef25 mnt]#
```

7.增加处理器管理结构

- 创建 `os/src/task/processor.rs`，实现处理器管理结构 `Processor`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/processor.rs
[root@7b99f7bbef25 mnt]#
```

8.创建初始进程

- 修改 `os/src/task/mod.rs`，接入 `pid`、`manager`、`processor` 模块，并创建初始进程 `INITPROC`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/mod.rs
[root@7b99f7bbef25 mnt]#
```

9.进程调度机制

- 修改 `os/src/task/mod.rs`，实现新的 `suspend_current_and_run_next()`。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/mod.rs
[root@7b99f7bbef25 mnt]#
```

10.进程的生成机制

- 修改 `os/src/mm/memory_set.rs`，为 `MapArea` 增加从已有逻辑段复制元信息的方法。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/mm/memory_set.rs`，为 `MemorySet` 增加复制已有用户地址空间的方法。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/mm/address.rs`，为 `PhysAddr` 增加 `get_mut()` 方法

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/address.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/mm/page_table.rs`，增加虚拟地址翻译、字符串翻译和可变引用翻译

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/page_table.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/mm/mod.rs`，导出新的地址翻译辅助函数。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/trap/mod.rs`，更新系统调用处理逻辑。`exec` 会替换当前进程地址空间，所以系统调用返回后需要重新获取 `Trap` 上下文。

```
[root@7b99f7bbef25 mnt]# vim os/src/trap/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/trap/mod.rs`，补全异常处理逻辑。系统调用返回后重新获取 `Trap` 上下文；应用发生访存或取指错误时，按错误退出码结束当前进程。

```
[root@7b99f7bbef25 mnt]# vim os/src/trap/mod.rs
[root@7b99f7bbef25 mnt]#
```

11.进程资源回收机制

- 修改 `os/src/task/mod.rs`，实现带退出码的 `exit_current_and_run_next`。进程退出时，需要把当前进程标记为 `Zombie`，记录退出码，并把子进程交给 `initproc` 接管

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/syscall/process.rs`，让 `sys_exit` 调用新的 `exit_current_and_run_next(exit_code)`，把应用退出码传给内核保存。

```
[root@7b99f7bbef25 mnt]# vim os/src/syscall/process.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/mm/memory_set.rs`，为 MemorySet 增加回收用户地址空间数据页的方法。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `os/src/main.rs`，把原来直接运行第一个任务的方式，改成先加入初始进程 `initproc`，再进入进程调度循环。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/main.rs
[root@7b99f7bbef25 mnt]#
```

12. 编译并运行验证

- 在 Docker 环境中进入 `os` 目录，重新编译并运行内核。

Bash

```
1 cd /mnt/os
2 make run
```

```
@7b99f7bbef25:/mnt/os
last 554 Physical Frames.
.text [0x80200000, 0x8020c000)
.rodata [0x8020c000, 0x80210000)
.data [0x80210000, 0x802c5000)
.bss [0x802c5000, 0x805d6000)
mapping .text section
mapping .rodata section
mapping .data section
mapping .bss section
mapping physical memory
[kernel] back to world!
remap_test passed!
after initproc!
/**** APPS ****
00power_3
01power_5
02power_7
03sleep
04addr_a
05addr_b
initproc
user_shell
*****/
Rust user shell
>>
```

- 输入一个已有的程序

Bash

```
1 00power_3
```

```
@7b99f7bbef25:/mnt/os
power_3 [10000/200000]
power_3 [20000/200000]
power_3 [30000/200000]
power_3 [40000/200000]
power_3 [50000/200000]
power_3 [60000/200000]
power_3 [70000/200000]
power_3 [80000/200000]
power_3 [90000/200000]
power_3 [100000/200000]
power_3 [110000/200000]
power_3 [120000/200000]
power_3 [130000/200000]
power_3 [140000/200000]
power_3 [150000/200000]
power_3 [160000/200000]
power_3 [170000/200000]
power_3 [180000/200000]
power_3 [190000/200000]
power_3 [200000/200000]
3^200000 = 871008973
Test power_3 OK!
[kernel] Application exited with code 0
Shell: Process 2 exited with code 0
>>
```

- 输入一个不存在的程序名

Bash

```
1 abc
```

```
>> abc
Error when executing!
[kernel] Application exited with code -4
Shell: Process 2 exited with code -4
>>
```

至此，具有进程管理功能的操作系统实现完成。

三.问题回答

1.分析应用的链接与加载是如何实现的；

本实验中，应用程序首先在 user 目录下单独编译，生成对应的 ELF 文件。内核在编译时通过 os/build.rs 扫描用户程序，并生成 link_app.S，把应用程序的 ELF 数据直接链接到内核镜像中。

Rust

```
1 .incbin "../user/target/riscv64gc-unknown-none-elf/release/app_name"
```

同时，build.rs 还会生成应用数量、应用起止地址表以及应用名称表 _app_names。这样内核运行时就可以根据应用名称找到对应的 ELF 数据。

Rust

```
1 .global _app_names
2 _app_names:
3     .string "initproc"
4     .string "user_shell"
```

应用加载主要由 loader.rs 和 memory_set.rs 完成。loader.rs 根据应用名称查找 ELF 数据，get_app_data_by_name() 会返回指定应用的 ELF 内容。

Rust

```
1 pub fn get_app_data_by_name(name: &str) -> Option<&'static [u8]>
```

真正创建应用地址空间时，内核调用 MemorySet::from_elf() 解析 ELF 文件。它会读取 ELF 的程序头，将代码段、数据段等映射到用户地址空间，并按照 ELF 中的权限设置页表权限。

Rust

```
1 let elf = xmas_elf::ElfFile::new(elf_data).unwrap();
```

加载时应用使用 Framed 映射，也就是为用户程序的虚拟页分配新的物理页帧，再把 ELF 中的数据复制进去。同时，内核还会为应用映射用户栈、TrapContext 页和跳板页。

因此，本实验的应用链接方式是“编译时打包进内核”，加载方式是“运行时根据 ELF 创建独立地址空间”。这样既能让内核直接访问应用数据，又能通过页表保证不同应用之间的地址空间隔离。

2.分析进程标识符、进程控制块是如何设计和实现的；

本实验中，每个进程通过 pid 进行唯一标识。pid 由 PidAllocator 统一分配，它内部用 current 记录下一个可分配 pid，用 recycled 保存已经回收的 pid。

Rust

```
1 pub struct PidAllocator {
2     current: usize,
3     recycled: Vec<usize>,
4 }
```

分配 pid 时，优先复用 recycled 中的 pid；没有可复用 pid 时，再从 current 分配新的 pid。每个 pid 用 PidHandle 保存，并通过 Drop 自动回收，避免进程结束后 pid 泄漏。

进程控制块由 TaskControlBlock 表示，主要保存 pid、内核栈以及进程内部状态。

Rust

```
1 pub struct TaskControlBlock {
2     pub pid: PidHandle,
3     pub kernel_stack: KernelStack,
4     inner: UPSafeCell<TaskControlBlockInner>,
5 }
```

其中 TaskControlBlockInner 保存进程的地址空间、任务上下文、运行状态、父子进程关系和退出码。父进程使用 Weak 指针，子进程使用 Arc 保存，避免循环引用。

因此，本实验通过 PidAllocator 实现 pid 的分配与回收，通过 TaskControlBlock 记录进程运行所需的信息，实现了进程的基本管理。

3.分析任务管理是如何实现的；

任务管理主要由 TaskManager 和 Processor 完成。TaskManager 负责维护就绪队列，内部使用 VecDeque<Arc<TaskControlBlock>> 保存所有处于就绪状态的进程。

Rust

```
1 pub struct TaskManager {
2     ready_queue: VecDeque<Arc<TaskControlBlock>>,
3 }
```

当创建新进程或进程让出 CPU 时，会调用 add_task() 将进程加入就绪队列。调度时通过 fetch_task() 从队首取出一个就绪进程运行。

Rust

```
1 pub fn add_task(task: Arc<TaskControlBlock>)  
2 pub fn fetch_task() -> Option<Arc<TaskControlBlock>>
```

Processor 用来表示当前处理器的执行状态，主要保存当前正在运行的进程和调度器上下文。

Rust

```
1 pub struct Processor {  
2     current: Option<Arc<TaskControlBlock>>,  
3     idle_task_cx: TaskContext,  
4 }
```

当系统运行时，run_tasks() 不断从就绪队列中取出任务，将其状态改为 Running，然后通过 __switch 切换到该进程执行。发生时钟中断或进程主动让出时，当前进程会重新进入就绪队列。

4.分析进程的调度、生成、以及进程资源的回收是如何实现的。

进程调度主要依靠时钟中断实现。发生时钟中断后，内核设置下一次时钟中断，并调用 suspend_current_and_run_next() 让当前进程让出 CPU。

Rust

```
1 Trap::Interrupt(Interrupt::SupervisorTimer) => {  
2     set_next_trigger();  
3     suspend_current_and_run_next();  
4 }
```

进程生成主要通过 fork 和 exec 完成。fork 会复制当前进程的地址空间和进程控制块，生成一个子进程，并把子进程加入就绪队列。子进程返回值被设置为 0，父进程返回子进程 pid。

Rust

```
1 pub fn fork(self: &Arc<Self>) -> Arc<Self>
```

exec 则会用新的 ELF 文件重新创建当前进程的地址空间，并更新 TrapContext，使当前进程从新程序入口开始执行。

Rust

```
1 pub fn exec(&self, elf_data: &[u8])
```

进程退出时，内核调用 `exit_current_and_run_next(exit_code)`，将当前进程状态改为 `Zombie`，记录退出码，并回收用户地址空间。它的子进程会交给 `initproc` 接管。

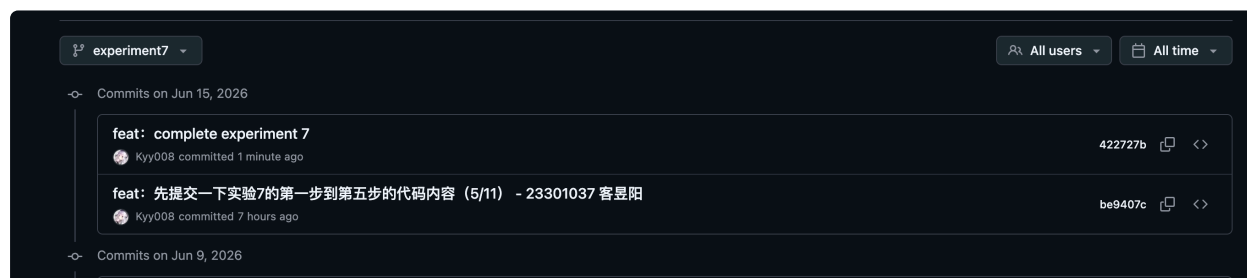
Rust

```
1 inner.task_status = TaskStatus::Zombie;  
2 inner.exit_code = exit_code;  
3 inner.memory_set.recycle_data_pages();
```

父进程通过 `waitpid` 回收已经退出的子进程。如果子进程已经是 `Zombie`，父进程会取得它的退出码，并从子进程列表中移除该进程。

因此，本实验通过时钟中断实现进程调度，通过 `fork` 和 `exec` 实现进程生成，通过 `exit` 和 `waitpid` 实现进程资源回收。

四.Git 提交截图



五.其他说明

无